# CONTROLS ENGINEERING TECHNICAL WORK SAMPLE

## Automated Ride Station Control & Fault Diagnostics
## Integrated Python Controller, HMI and Ladder Instruction Monitoring

**Prepared for:** Universal Orlando — Controls Engineer application

**Prepared by:** Angel Seoane

**Document baseline:** October 2026

## Project at a glance

| Area | Demonstrated capability |
|---|---|
| Control model | Python RideStation discrete-state controller; eight Boolean inputs; diagnostic fault latching |
| Operator interface | Browser HMI connected to Python through a localhost HTTP/JSON API |
| Runtime | Manual and automatic 500 ms scans; HMI refresh approximately 250 ms |
| Diagnostics | Active faults, latched alarms, scan counter, event history and fault reset |
| Logic visualization | Animated ladder rungs; XIC, XIO, OTE, OTL, OTU and TON status monitoring |

**Scope statement.** Independently developed educational simulation. Not affiliated with Universal Orlando. No physical PLC, ride equipment, certified safety controller, or real-time deterministic control. Phase 5 executable ladder engine is not included.

## 1 | Engineering objective and design approach

Demonstrate controls-engineering reasoning through an observable, testable single-train station model. The work emphasizes state sequencing, dispatch permissives, fault identification, operator interaction, testability and clear boundaries between controller logic and visualization.

## 2 | System architecture

| Layer | Implementation | Role |
|---|---|---|
| Simulated field I/O | Eight Boolean inputs | Inject and monitor modeled device conditions |
| Control logic | ride_sim.py / RideStation | Owns states, transitions, faults, and event history |
| Application server | api_server.py | Provides JSON API, shared controller access and automatic scanning |
| Operator HMI | dashboard.html | Commands, input toggles, live state and diagnostics |
| Instruction monitor | Phase 4 monitoring model | Read-only rung status and scan-sampled timer indications |

## Data and command flow

**Operator action** → HTTP request → Python API → RideStation command / scan → state snapshot → HMI refresh. A background worker performs automatic scans at the configured 500 ms interval; the browser polls status approximately every 250 ms.

## Architecture decision

The Python RideStation is the authoritative controller model. The browser no longer maintains a separate duplicate state machine. Phase 3 and 4 rung displays derive indications from controller data; they do not write control outputs or execute a PLC program.

## 3 | Control philosophy and I/O schedule

Nominal states: IDLE → READY → DISPATCHED → AWAY. A modeled diagnostic failure transitions the controller to FAULT. A reset requires diagnostic faults to be restored and all eight permissives to be true.

| Input tag | Purpose |
|---|---|
| train_present | Train detected in station |
| gate_closed | Station gate closed |
| restraints_locked | Restraints verified locked |
| block_clear | Downstream block available |
| drive_ready | Drive readiness indication |
| estop_healthy | Illustrative E-stop health status |
| plc_comms_ok | Illustrative controller communications status |
| sensor_consistent | Simulated sensor agreement |

## Dispatch permissive

TRAIN_PRESENT AND GATE_CLOSED AND RESTRAINTS_LOCKED AND BLOCK_CLEAR AND DRIVE_READY AND ESTOP_HEALTHY AND PLC_COMMS_OK AND SENSOR_CONSISTENT.

## Fault classification

The four modeled diagnostic-fault inputs are **estop_healthy**, **plc_comms_ok**, **sensor_consistent**, and **drive_ready**. Gate, restraint, train-presence, and block-clear failures inhibit dispatch without automatically latching a diagnostic alarm in this simplified model.

## 4 | Sequence of operation and operator interaction

| Action/condition | Controller response |
|---|---|
| Initial valid scan | IDLE → READY |
| Dispatch request while READY and permitted | READY → DISPATCHED |
| Following scan | DISPATCHED → AWAY; train_present=FALSE; block_clear=FALSE |
| Return simulation in AWAY with block_clear=TRUE | AWAY → IDLE; train_present=TRUE |
| Next scan with all permissives TRUE | IDLE → READY |
| Diagnostic failure on scan | State becomes FAULT; associated alarm latched |
| Restore input without reset | Fault state / latched alarm remains |
| Reset with all permissives restored | Fault cleared; controller returns to READY |

## Arrival flow detail

The operator must set block_clear to TRUE while the controller is AWAY before issuing SIMULATE ARRIVAL. This is an educational simplification; the model does not represent separate approach-zone occupancy, station approach sensors, braking, or train stopping dynamics.

## Automatic scan and HMI behavior

In AUTO mode, the controller scans at a nominal 500 ms interval. The HMI polls for updates approximately every 250 ms, displaying mode, scan count, permissives, active faults, latched alarms, and the recent scan log. The IDLE state after arrival may be short-lived because the next scan can immediately produce READY.

## 5 | Fault isolation and verification

| Fault injection | Expected behavior / diagnostic interpretation |
|---|---|
| plc_comms_ok = FALSE | FAULT; PLC_COMMUNICATION_LOSS latched; examine communications and tag path |
| sensor_consistent = FALSE | FAULT; SENSOR_DISAGREEMENT latched; compare signal sources |
| drive_ready = FALSE | FAULT; DRIVE_NOT_READY latched; inspect modeled drive status |
| estop_healthy = FALSE | FAULT; E_STOP_CIRCUIT_OPEN latched |
| gate_closed = FALSE | Dispatch inhibited; no diagnostic latch by itself |
| block_clear = FALSE | Dispatch inhibited; arrival request not accepted |

## Test evidence and coverage

The original Python reference controller work sample reported **10 unit tests passed** and **six deterministic scenario traces**, including dispatch inhibition, diagnostic latching, reset behavior and return conditions. Those historical results validate the original RideStation test scope, not complete end-to-end automated testing of the Phase 4 browser UI.

## Suggested Phase 4 acceptance walkthrough

1. Restart and scan to READY. 2. Dispatch, scan to AWAY and observe timer status. 3. Restore block_clear and simulate arrival. 4. Inject communications failure and confirm fault latch. 5. Restore communications and reset. 6. Start/stop AUTO and confirm the scan counter changes only while enabled.

## Engineering interpretation

The project provides observable cause-and-effect relationships: a missing permissive blocks a command; a diagnostic failure latches; signal restoration alone does not clear a latched fault; the operator must perform a valid reset.

## 6 | Phase-by-phase implementation

| Phase | Implemented capability |
|---|---|
| Phase 1 — Controller and HMI | RideStation states, eight inputs, fault logic, CLI; connected HTML dashboard through local API |
| Phase 2 — Scan automation | 500 ms AUTO/MANUAL scan modes, 250 ms HMI refresh, scan counter and fault panels |
| Phase 3 — Ladder monitor | Read-only visual rungs, energized/de-energized contacts and animated power flow |
| Phase 4 — Instruction monitor | Instruction-status panel for XIC/XIO/OTE/OTL/OTU and 2-second AWAY_DWELL TON |

## Phase 4 instruction semantics

| Instruction | Phase 4 behavior |
|---|---|
| XIC / XIO | Show whether a TRUE / FALSE tag condition would pass logical continuity |
| OTE | Shows coil-like non-retentive output indication |
| OTL / OTU | Reflect Python alarm latch and alarm clear/reset conditions |
| TON | AWAY_DWELL informational timer: PRE=2000 ms; EN, TT, DN and ACC shown |

**Critical implementation boundary:** Phase 4 instructions are displayed as monitored semantics. They are not a rung-order scan-executed ladder engine. The AWAY_DWELL.DN bit does not cause train movement or enforce an interlock.

## 7 | HMI evidence

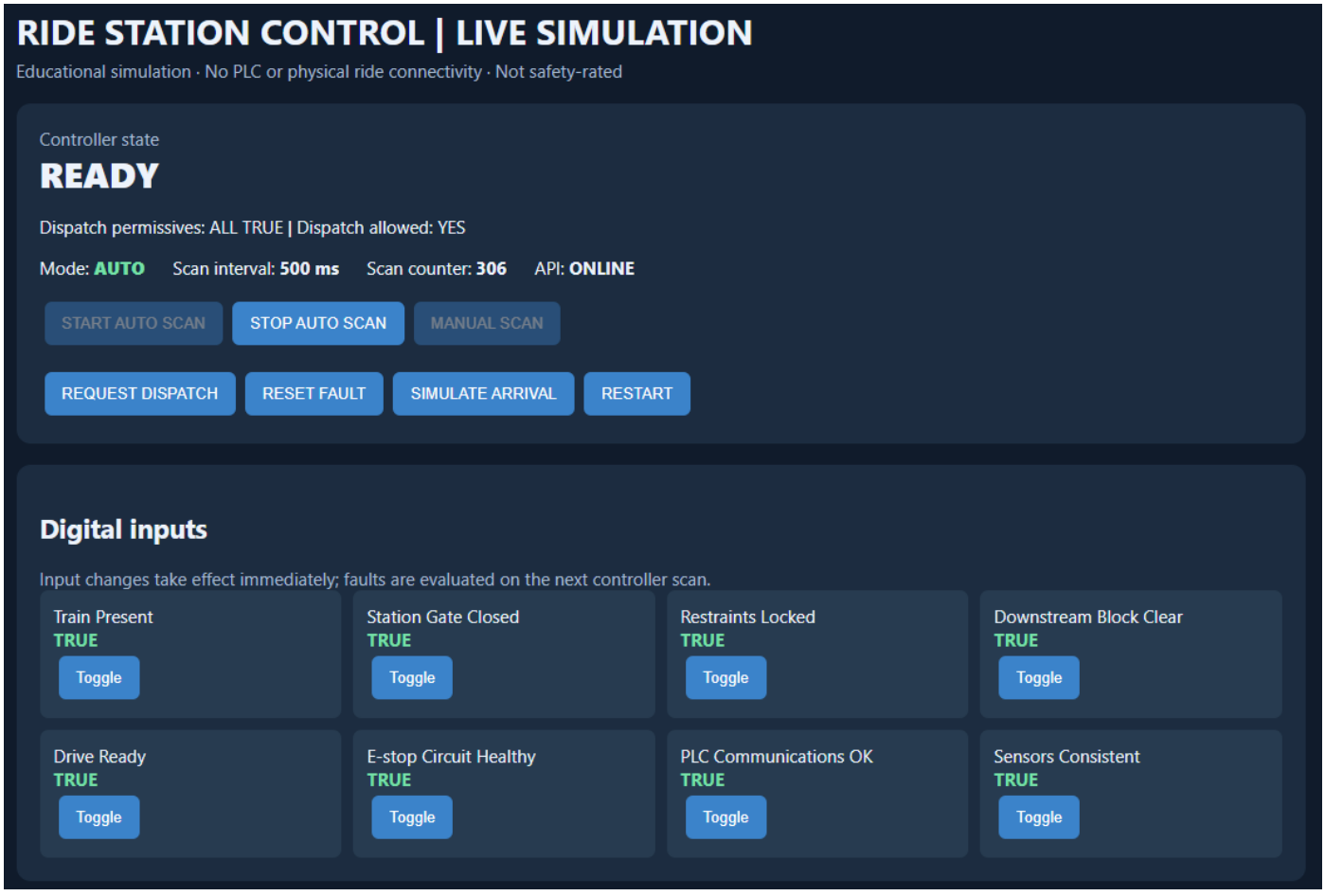
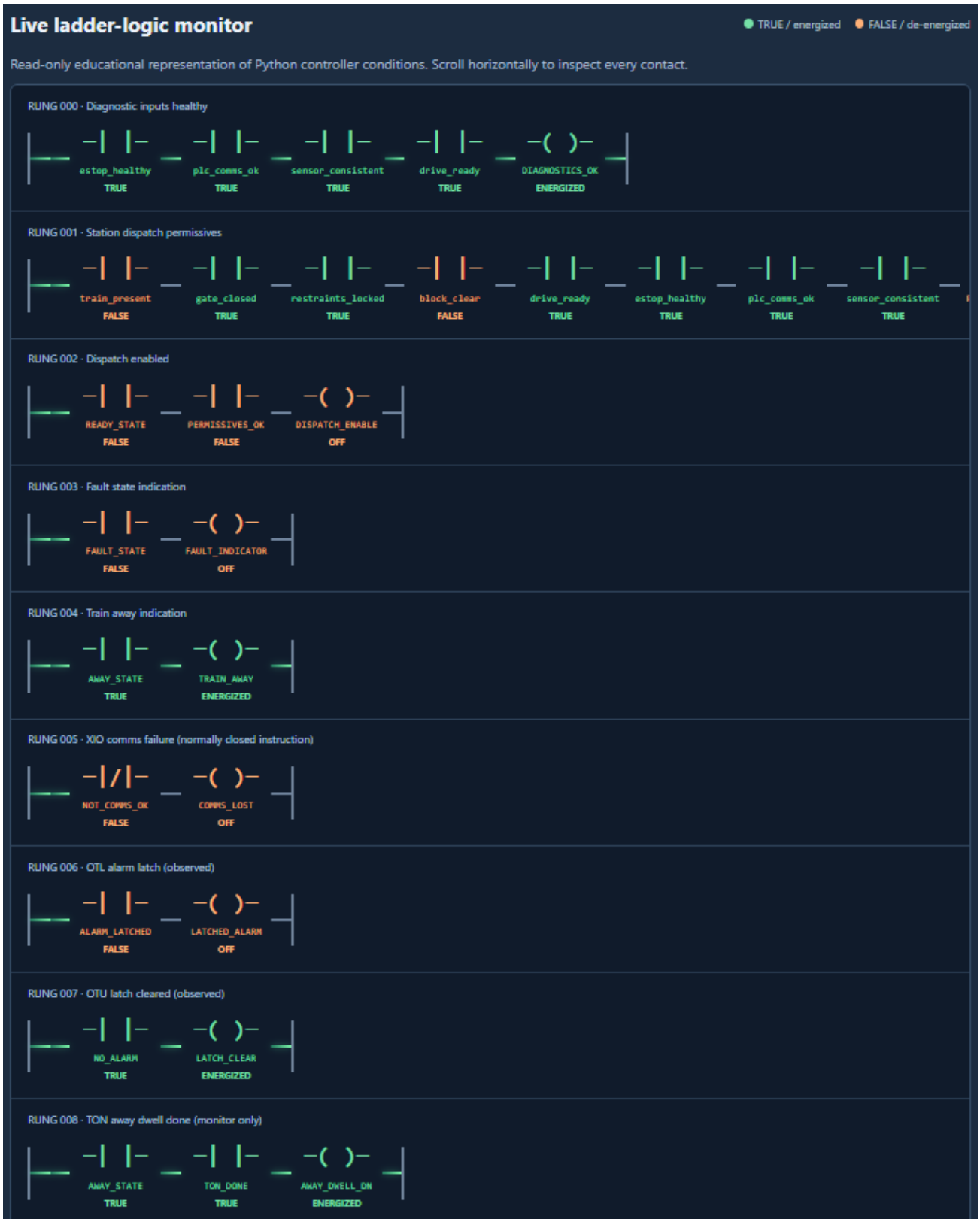



**Figure 1. Live Ride Station HMI — Automatic Scan Operation**

Browser-based operator interface connected to the Python ride-control simulator through a local REST API. The controller is in READY state with all eight dispatch permissives satisfied. Automatic scanning is enabled at 500 ms intervals, with 306 completed scans and dispatch authorization active.




## Figure 2. Live Ladder-Logic Monitoring — Train Away State

Read-only ladder visualization showing simulated controller conditions across nine rungs. Green indicates TRUE or energized conditions; amber indicates FALSE or de-energized conditions. The train-away indication and AWAY_DWELL completion are energized, while dispatch remains inhibited because the train is absent and the downstream block is not clear.

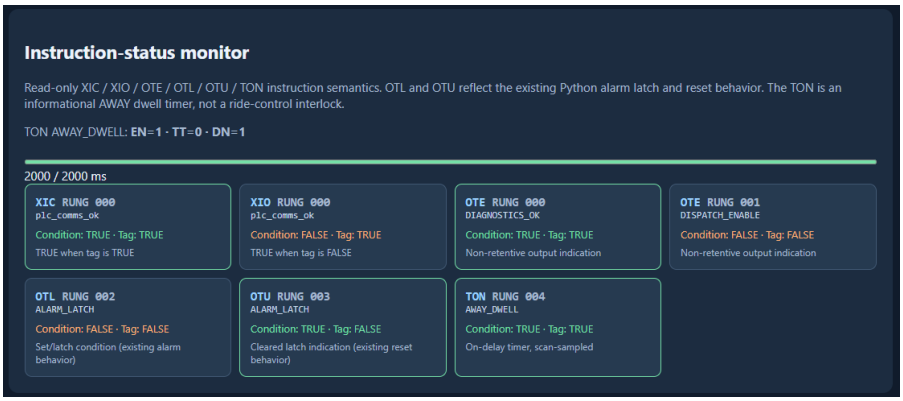
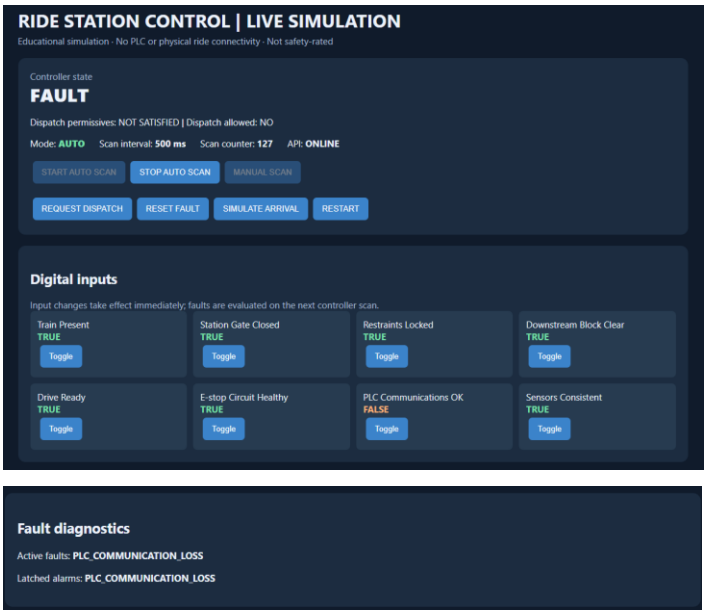



**Figure 3. Ladder Instruction-Status Monitor — TON Completion.**

Read-only visualization of XIC, XIO, OTE, OTL, OTU, and TON instruction semantics. The AWAY_DWELL on-delay timer has reached its 2,000 ms preset, with Enable (EN) TRUE, Timer Timing (TT) FALSE, and Done (DN) TRUE. The panel also displays diagnostic, dispatch-enable, and alarm-latch conditions derived from the Python controller.

## 8 | HMI evidence — fault diagnostics




## Figure 4. PLC Communication Failure — Controller Fault Detection and Alarm Latching.

The simulated PLC Communications OK input is set to FALSE, causing the Python controller to transition to FAULT during the next automatic scan. Dispatch is inhibited, and the diagnostic system identifies PLC_COMMUNICATION_LOSS as both an active fault and a latched alarm. The controller continues scanning at 500 ms intervals while the local API remains online. Restoring communications alone does not clear the latched fault; an operator reset is required after all permissives are satisfied.

## 9 | Professional context and project limitations

Prior manufacturing experience includes over three years of Wonderware InTouch SCADA/HMI development and troubleshooting, Allen-Bradley PLC integration, communication and tag diagnostics, and limited RSLogix 500 online ladder monitoring. That prior role did **not** involve writing or modifying PLC ladder programs.

This independently developed project is an educational portfolio exercise, not a commissioned Universal Orlando system. It has no safety-rated PLC, redundant safety inputs, certified interlocks, real-time scheduling guarantee, physical ride interface, validated braking, block-control dynamics, or motion control. It is not intended for deployment to actual equipment.